Internship Project Plan: Planning-Application Lead Engine (Prototype)

Duration: 8 weeks | **Stack:** Python, LLM APIs | **Scope:** One UK council

---

Project Overview

Every week, UK councils publish hundreds of planning applications and approvals through their online planning portals. Buried in that data are high-value sales leads: a granted extension means work for scaffolders, kitchen fitters, skip hire firms, party-wall surveyors, and architects. Existing products (Searchland, LandTech) serve property developers; nobody packages this data for the tradespeople downstream of an approval — a market that would pay £X/month for relevant, local, ready-to-action leads.

Your project is to build a working prototype of that pipeline for **one UK council**: extract planning applications, store them in a clean, LLM-friendly format, use an LLM to classify each application by the trades it's relevant to, and present the results as a simple, sellable lead feed.

Success looks like: by the end of week 8, we can point at a single command (or scheduled job) that pulls the latest applications from the council portal, enriches and classifies them, and produces a weekly "lead digest" that a scaffolder or kitchen fitter would recognise as genuinely useful.

Phase 1 — Discovery & Design (approx. weeks 1–2)

Goal: Understand the data source and design the system before writing pipeline code.

- Select the target council together (criteria: portal accessibility, volume of applications, proximity to trades we could realistically sell to). Most councils use one of a handful of portal systems (e.g. Idox "Public Access"), which matters for extraction strategy.

- Explore the portal manually: what fields exist per application (reference number, address, description, applicant/agent, status, decision date, documents)? What's the difference between *submitted*, *validated*, and *decided/granted* applications? Granted applications are the strongest lead signal.

- Check terms of use / robots.txt and confirm our extraction approach is polite and legitimate (rate-limited, cached, attribution where required). Also check whether the council offers any structured feed or API before defaulting to scraping.

- Draft a one-page technical design: extraction method, storage format, classification approach, and how the pieces connect.

- Define the initial **trade taxonomy** — the list of trades we'll classify against (e.g. scaffolding, groundworks, roofing, kitchens/bathrooms, electrical, skip hire, architecture, party-wall surveying), with a short description of what triggers each.

Deliverable: Technical design document + trade taxonomy, reviewed and agreed before Phase 2.

Phase 2 — Extraction Pipeline (approx. weeks 2–4)

Goal: Reliable, repeatable extraction of planning applications into raw storage.

- Build a Python extractor (e.g. `requests` + `BeautifulSoup`/`Playwright` as needed) that pulls new and recently decided applications from the portal.
- Make it **incremental and idempotent**: it should be runnable daily, fetch only what's new or changed, never duplicate records, and recover gracefully from portal errors.
- Store each application as a structured record. Recommended approach: a lightweight database or JSON store as the source of truth, plus **one Markdown file per application** (clean headings, key fields, full description text) as the LLM-ready layer — Markdown is cheap in tokens, human-readable, and easy to inspect.
- Add basic logging and a simple run report ("fetched 43 applications, 5 new, 2 status changes").

Deliverable: A working extractor with 4+ weeks of historical data for the council, plus a short README so someone else could run it.

Phase 3 — LLM Classification & Enrichment (approx. weeks 4–6)

Goal: Turn raw applications into trade-tagged leads.

- Build a classification step that sends each application's Markdown to an LLM and returns structured output (JSON): relevant trades (from the taxonomy), a confidence score per trade, an estimated project scale (small/medium/large), a plain-English one-line summary, and the best contact route (applicant vs. agent).
- Focus on prompt quality and cost efficiency: batch where possible, use a cheap model, and only classify applications worth classifying (e.g. skip tree works and advertisements early with simple keyword filters).
- Build a small **evaluation set**: hand-label ~50 applications yourself, then measure how well the LLM agrees. Iterate on the prompt until accuracy is credible — this is the core of the product's value, so it deserves real attention.
- Store classification results alongside each application record.

Deliverable: Classified dataset + an accuracy write-up (even a simple one) showing how the classifier performs against your hand labels.

Phase 4 — Lead Output & Delivery (approx. weeks 6–7)

Goal: Package the data as something a customer would pay for.

- Build the "lead digest" output: for a given trade and postcode/radius, generate a weekly digest (email-style HTML or Markdown) listing relevant approvals with summary, address, decision date, and contact route.

- Add basic filtering: by trade, by application status (granted only vs. all), by area.

- Optional stretch (if time and skills allow): a minimal web view (e.g. Streamlit or FastAPI + simple frontend) so we can demo the product live rather than showing files.

Deliverable: Sample digests for 2–3 different trades that we could put in front of a real tradesperson for feedback.

Phase 5 — Hardening, Documentation & Handover (approx. week 8)

Goal: Leave behind something we can run without you.

- Schedule the full pipeline (extract → classify → digest) to run automatically (cron or a simple scheduler), with failure alerts.

- Write handover documentation: architecture overview, how to run/repair each stage, known limitations, and rough per-application LLM cost.

- Prepare and deliver a final presentation:` a live demo, what worked, what didn't, honest accuracy numbers, and your recommendations for scaling to a second council (what generalises, what was council-specific).

Deliverable: Documented, scheduled prototype + final presentation.

Working Practices & Expectations

- **Code:** All work in a Git repository with regular commits; Python, sensibly structured, with a README per component.

- **Check-ins:** Short weekly review of progress against the current phase; raise blockers early rather than working around them silently.

- **Costs:** LLM and hosting spend must stay minimal — track per-run cost from Phase 3 onwards; cost-per-lead is a key business metric, not an afterthought.

- **Scope discipline:** One council, done well, beats three done badly. Anything beyond scope goes on a "next version" list rather than into the build.

- **Data care:** Applicant/agent details are personal data — we store only what's published, use it responsibly, and note any GDPR considerations in the handover doc.

What You'll Get Out of This

A complete, end-to-end build you fully own: web data extraction, pipeline engineering, practical LLM integration with real evaluation, and turning raw data into a commercial product. If the prototype validates, this becomes the foundation of a real revenue line — and you'll have built version one.

Happy to adjust the phase weighting, add a specific council recommendation, or turn this into a PDF/Word document for sharing.

View or comment on planning applications | Westminster City Council

Search Results